REVIEW SHEET

The maths behind the digit classifier

Every equation used in `tinynet`, what it means, where it comes from, and which line of code computes it.

How to use this sheet. Sections follow the order of the tutorial. Each one ends with the file and function that implements it, shown like this: `layers.py · Dense.backward`
The one-page summary at the end lists every formula on its own, for quick lookup.

Contents

1 Notation 2

2 Preparing the data 2

3 The Dense layer 3

4 Activation functions 3

5 Dropout 4

6 From scores to a loss 4

7 Backpropagation 5

8 Checking gradients numerically 6

9 Optimisers 6

10 Training and evaluation 7

11 One-page summary 9

1 Notation

Symbol	Meaning
n	number of pictures in a batch (32 while training)
d	number of input features: 64 pixels
K	number of classes: 10 digits
$\mathbf{x} \in \mathbb{R}^d$	one picture, flattened to a row of 64 numbers
$X \in \mathbb{R}^{n \times d}$	a batch: one picture per row (`X` in the code)
$y \in \{0, \ldots, 9\}$	the correct digit for a picture (`y`)
$\mathbf{t} \in \{0,1\}^K$	the one-hot target: $t_k = 1$ if $k = y$, else 0
W, $\mathbf{b}$	weights and biases of a Dense layer (`self.W` , `self.b`)
$\mathbf{z} \in \mathbb{R}^K$	logits: the ten raw scores the network outputs
$\mathbf{p} \in \mathbb{R}^K$	probabilities from softmax, $\sum_k p_k = 1$
L	the loss, a single number
$\frac{\partial L}{\partial W}$	the gradient: an array the same shape as W (`self.dW`)
η	learning rate (`lr`)
$\odot$	element-wise multiplication (`*` in NumPy)
AB	matrix multiplication (`A @ B` in NumPy)

Vectors are rows. The code stores one example per *row*, so this sheet writes a layer as $\mathbf{x}W$, not $W\mathbf{x}$. Many textbooks use columns; the maths is the same, transposed.

2 Preparing the data

2.1 Scaling

Pixels arrive as integers 0 to 16. Dividing by 16 maps them into $[0, 1]$:

$$x_j \leftarrow \frac{x_j}{16}.$$

Inputs of order 1 match the assumption behind the weight initialisation in Section 3.3: signals start neither tiny nor huge. `data.py · load_data`

2.2 One-hot encoding

A label y becomes a vector with a single 1:

$$t_k = \begin{cases} 1 & k = y \\ 0 & \text{otherwise} \end{cases} \qquad \text{e.g. } y = 3 \;\mapsto\; \mathbf{t} = (0,0,0,1,0,0,0,0,0,0).$$

This lets the label be compared directly with the network's ten probabilities. `data.py · one_hot`

2.3 Splits

The 1,797 pictures are shuffled with a fixed seed, then split 70/15/15: training (1,257) to learn from, validation (269) to make decisions, test (271) to report once. Shuffling X and y with the *same* permutation keeps each picture paired with its label.

3 The Dense layer

3.1 Forward pass

For a single picture, output j is a weighted sum of the inputs plus a bias:

$$\text{out}_j = \sum_{i=1}^{n_{\text{in}}} x_i W_{ij} + b_j.$$

For a whole batch at once this is one matrix product, with $\mathbf{b}$ added to every row (NumPy's *broadcasting*):

$$\underbrace{O}_{n\times n_{\text{out}}} = \underbrace{X}_{n\times n_{\text{in}}}\ \underbrace{W}_{n_{\text{in}}\times n_{\text{out}}} + \underbrace{\mathbf{b}}_{1\times n_{\text{out}}}.$$

`layers.py · Dense.forward`

> **The shape rule**
> Matrix multiplication needs the inner sizes to agree: $(a \times k)(k \times c) = (a \times c)$. Almost every bug in a hand-written network shows up as a shape that breaks this rule.

3.2 Parameter count

A Dense layer from n_{in} to n_{out} has $n_{\text{in}}n_{\text{out}}$ weights and n_{out} biases. For the $64 \to 64 \to 32 \to 10$ network:

$$(64 \cdot 64 + 64) + (64 \cdot 32 + 32) + (32 \cdot 10 + 10) = 4160 + 2080 + 330 = 6570.$$

3.3 Why the weights start random, and how large

If every weight started equal, every neuron in a layer would compute the same output, receive the same gradient, and stay identical for ever (the *symmetry* problem). Random starting values break the tie.

The size matters too. Assume inputs x_i and weights W_{ij} are independent with mean zero. Then the variance of a sum of n_{in} terms adds up:

$$\text{Var}(\text{out}_j) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(x_i W_{ij}) = n_{\text{in}}\ \text{Var}(x)\ \text{Var}(W).$$

To keep the signal the same size from layer to layer we want $n_{\text{in}}\,\text{Var}(W) \approx 1$. ReLU (Section 4.2) zeroes about half of its inputs, halving the signal's second moment, so we double it back:

$$\text{Var}(W) = \frac{2}{n_{\text{in}}} \qquad \Longrightarrow \qquad W_{ij} \sim \mathcal{N}\left(0,\ \tfrac{2}{n_{\text{in}}}\right) \quad \text{(mean 0, variance } 2/n_{\text{in}}\text{)}.$$

This is **He initialisation**. The code draws with standard deviation $\sqrt{2/n_{\text{in}}}$; biases start at zero.
`layers.py · Dense.__init__`

4 Activation functions

4.1 Why a non-linearity is needed

Two Dense layers with nothing between them collapse into one:

$$(XW_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2 = X\underbrace{(W_1W_2)}_{W'} + \underbrace{(\mathbf{b}_1W_2 + \mathbf{b}_2)}_{\mathbf{b}'}.$$

However many are stacked, the result is still a single linear map. A non-linear function between layers is what lets depth add power.

4.2 ReLU

$$\mathrm{ReLU}(u) = \max(0, u), \qquad \frac{d}{du}\mathrm{ReLU}(u) = \begin{cases} 1 & u > 0 \\ 0 & u \le 0. \end{cases}$$

(The derivative at exactly 0 is undefined; the code uses 0, which never matters in practice.) Applied element-wise to a whole array, the backward pass multiplies the incoming gradient by the *mask* $M = [\,U > 0\,]$:

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial O} \odot M.$$

`layers.py · ReLU`

> **Dead neurons**
> If a neuron's input is negative for every picture, its mask is always 0, so its gradient is always 0 and it can never recover. A learning rate that is too large can push most neurons into this state, which is what happened in the `Adam(lr=0.5)` experiment.

5 Dropout

While training, each value is kept with probability $q = 1 - \text{rate}$ and zeroed otherwise, and the survivors are divided by q. With $m_i \sim \mathrm{Bernoulli}(q)$:

$$\mathrm{out}_i = \frac{m_i}{q}\, x_i, \qquad \mathbb{E}[\mathrm{out}_i] = \frac{\mathbb{E}[m_i]}{q}\, x_i = \frac{q}{q}\, x_i = x_i.$$

Because the expected output already equals the input, nothing needs rescaling at test time: dropout simply returns x unchanged. (This version is called *inverted dropout*.) The backward pass uses the same mask: $\frac{\partial L}{\partial x_i} = \frac{m_i}{q}\frac{\partial L}{\partial \mathrm{out}_i}$.

`layers.py · Dropout`

6 From scores to a loss

6.1 Softmax

Turns ten arbitrary scores into ten positive numbers that sum to one:

$$p_k = \mathrm{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

The stability trick. Adding the same constant c to every score changes nothing:

$$\frac{e^{z_k - c}}{\sum_j e^{z_j - c}} = \frac{e^{-c}\, e^{z_k}}{e^{-c} \sum_j e^{z_j}} = \frac{e^{z_k}}{\sum_j e^{z_j}}.$$

Choosing $c = \max_j z_j$ makes the largest exponent $e^0 = 1$ and every other one at most 1, so `np.exp` can never overflow to `inf`.

`losses.py · softmax`

6.2 Cross-entropy

The loss for one picture is minus the log of the probability given to the correct digit y:

$$\ell = -\log p_y = -\sum_{k=1}^{K} t_k \log p_k,$$

(the two forms agree because $\mathbf{t}$ is one-hot). The batch loss is the mean, $L = \frac{1}{n}\sum_{i=1}^{n} \ell^{(i)}$.

p_y (probability on the right answer)	0.99	0.5	0.1	0.01
$-\ln p_y$	0.01	0.69	2.30	4.61

The number to remember: $\ln 10 \approx 2.3026$

A network that knows nothing gives every digit $p_k = \frac{1}{10}$, so its loss is $-\ln \frac{1}{10} = \ln 10$. A starting loss near 2.30 means the initialisation is sane; a loss stuck there means nothing is learning.

`losses.py · cross_entropy`

6.3 The gradient of softmax + cross-entropy

Substitute the softmax into the loss for one picture:

$$\ell = -\log \frac{e^{z_y}}{\sum_j e^{z_j}} = -z_y + \log \sum_j e^{z_j}.$$

Differentiate with respect to one score z_k. The first term gives -1 if $k = y$ and 0 otherwise, which is $-t_k$. The second term gives

$$\frac{\partial}{\partial z_k} \log \sum_j e^{z_j} = \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k.$$

So

$$\boxed{\frac{\partial \ell}{\partial z_k} = p_k - t_k} \qquad \text{and for the batch mean} \qquad \frac{\partial L}{\partial Z} = \frac{1}{n}(P - T).$$

Predicted minus target: positive for wrong digits (push their scores down), negative for the right one (push it up), and larger the bigger the mistake. The $\frac{1}{n}$ comes from averaging; it is the `/ n` in the code.
`losses.py · cross_entropy`

7 Backpropagation

7.1 The chain rule, one layer at a time

The network is a chain $X \to f_1 \to f_2 \to \cdots \to f_m \to L$. For any layer with input U and output $O = f(U)$, the chain rule says

$$\underbrace{\frac{\partial L}{\partial U}}_{\text{returned by } \texttt{backward}} = \underbrace{\frac{\partial L}{\partial O}}_{\text{received by } \texttt{backward}} \cdot \frac{\partial O}{\partial U}.$$

Each layer only needs its own local derivative $\frac{\partial O}{\partial U}$. Starting from $\frac{\partial L}{\partial Z} = \frac{1}{n}(P - T)$ and calling `backward` on each layer in reverse order delivers every gradient in the network in one sweep.
`model.py · Network.backward`

7.2 The three Dense-layer gradients

Write $G = \frac{\partial L}{\partial O}$ for the incoming gradient, shape $n \times n_{\text{out}}$. Because $O_{aj} = \sum_i X_{ai} W_{ij} + b_j$:

Weights. W_{ij} affects only column j of the output, at rate X_{ai} for each picture a:

$$\frac{\partial L}{\partial W_{ij}} = \sum_{a=1}^{n} X_{ai}\, G_{aj} \quad \Longrightarrow \quad \boxed{\frac{\partial L}{\partial W} = X^{\mathsf{T}} G} \quad (n_{\text{in}} \times n)(n \times n_{\text{out}}).$$

Biases. b_j shifts column j by the same amount for every picture:

$$\frac{\partial L}{\partial b_j} = \sum_{a=1}^{n} G_{aj} \quad \Longrightarrow \quad \boxed{\frac{\partial L}{\partial \mathbf{b}} = \textstyle\sum_{\text{rows}} G} \quad (\texttt{grad.sum(axis=0)}).$$

Inputs. X_{ai} feeds every output j of picture a, through weight W_{ij}:

$$\frac{\partial L}{\partial X_{ai}} = \sum_{j=1}^{n_{\text{out}}} G_{aj} W_{ij} \quad \Longrightarrow \quad \boxed{\frac{\partial L}{\partial X} = G\,W^{\mathsf{T}}} \quad (n \times n_{\text{out}})(n_{\text{out}} \times n_{\text{in}}).$$

Each gradient has the shape of the thing it is the gradient *of*: a free check. `layers.py · Dense.backward`

7.3 Why backprop is fast

Computing all P gradients one at a time by nudging (Section 8) costs about $2P$ forward passes. Backpropagation gets all of them for the cost of one forward and one backward pass, roughly two or three forward passes. For the 6,570 weights here that is over two thousand times faster, and the gap grows with the size of the network.

8 Checking gradients numerically

The definition of a derivative, taken symmetrically:

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}.$$

Why the symmetric version. A Taylor expansion gives $L(w \pm \varepsilon) = L(w) \pm \varepsilon L'(w) + \frac{\varepsilon^2}{2} L''(w) \pm \frac{\varepsilon^3}{6} L'''(w) + \ldots$ Subtracting, the even terms cancel:

$$\frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon} = L'(w) + O(\varepsilon^2),$$

so the error is about $\varepsilon^2 = 10^{-10}$, compared with $O(\varepsilon)$ for the one-sided $\frac{L(w+\varepsilon)-L(w)}{\varepsilon}$. Rounding in floating point limits how small ε can usefully be.

Reading the result. Differences around 10^{-9} (what the tutorial shows) mean the backward pass is correct; around 10^{-2} or more means a bug. `gradcheck.py · numerical_grad`

9 Optimisers

9.1 Gradient descent

Step each weight against its gradient:

$$w \leftarrow w - \eta\, g, \qquad g = \frac{\partial L}{\partial w}.$$

For a small enough step, $L(w - \eta g) \approx L(w) - \eta\, g^2 < L(w)$: the loss goes down. Using a mini-batch instead of all the data gives a noisy but unbiased estimate of g; that is the "stochastic" in SGD.

The learning rate's limit. On a bowl $L = \frac{1}{2} a w^2$ (gradient aw), one step gives $w \leftarrow (1 - \eta a)\, w$. This shrinks only if $|1 - \eta a| < 1$, i.e. $\eta < 2/a$. The steepest direction sets the limit, and the shallow directions then crawl: exactly what `check_optim.py` shows with steepnesses 1 and 100.

9.2 Momentum

Keep a running velocity:

$$v \leftarrow \mu v - \eta g, \qquad w \leftarrow w + v.$$

If g stays constant, v settles where $v = \mu v - \eta g$, i.e.

$$v_\infty = -\frac{\eta}{1-\mu}\, g,$$

so with $\mu = 0.9$ consistent gradients move the weights **ten times** faster. Gradients that flip sign largely cancel. With $\mu = 0$ this is plain gradient descent. `optim.py · SGD.step`

9.3 Adam

Two running averages per weight, a step count t, and a per-weight step size:

$$\begin{aligned}
m &\leftarrow \beta_1 m + (1-\beta_1)\, g && \text{average gradient } (\beta_1 = 0.9)\\
v &\leftarrow \beta_2 v + (1-\beta_2)\, g^2 && \text{average squared gradient } (\beta_2 = 0.999)\\
\hat{m} &= \frac{m}{1-\beta_1^t}, \qquad \hat{v} = \frac{v}{1-\beta_2^t} && \text{bias correction}\\
w &\leftarrow w - \eta\, \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} && \epsilon = 10^{-8}
\end{aligned}$$

Bias correction. With $m_0 = 0$, unrolling the average gives $m_t = (1-\beta_1)\sum_{s=1}^{t} \beta_1^{t-s} g_s$. If the gradients are all equal to g, this geometric sum is $m_t = (1-\beta_1^t)\, g$, too small early on. Dividing by $1-\beta_1^t$ recovers exactly g. At $t = 1$: $m_1 = 0.1g$, and $0.1g/(1-0.9) = g$.

Why the step is about η. If a weight's gradients are consistently g, then $\hat{m} \approx g$ and $\sqrt{\hat{v}} \approx |g|$, so the step is about η in size whatever the scale of g. Noisy gradients make $|\hat{m}|$ smaller than $\sqrt{\hat{v}}$, and the step shrinks. This is why Adam is far less sensitive to the learning rate than SGD. `optim.py · Adam.step`

> **The update must happen in place**
>
> The arithmetic is $w \leftarrow w - \Delta$, but in code it must be `p -= ...` or `p[...] = ...`. Writing `p = p - ...` creates a new array and leaves the layer's weights untouched: the maths is right, and nothing learns.

10 Training and evaluation

10.1 Epochs and batches

One epoch is one pass over the training data in mini-batches of 32: $\lceil 1257/32 \rceil = 40$ updates, the last batch holding $1257 - 39 \cdot 32 = 9$ pictures. The epoch's training loss is the average weighted by batch size:

$$L_{\text{train}} = \frac{1}{N} \sum_{\text{batches } B} |B|\, L_B,$$

so the small final batch counts in proportion to its size. `train.py · fit`

10.2 Early stopping

After each epoch compute the validation loss L_{val}. Keep a copy of the weights whenever it reaches a new minimum; stop once it has failed to improve for `patience` epochs in a row; then restore the best copy. The weights you keep are

$$w^\star = w^{(e^\star)}, \qquad e^\star = \arg\min_e L_{\text{val}}^{(e)}.$$

10.3 Accuracy and the confusion matrix

$$\text{prediction} = \arg\max_k z_k, \qquad \text{accuracy} = \frac{1}{N}\sum_{i=1}^{N} [\,\hat{y}^{(i)} = y^{(i)}\,],$$

where $[\cdot]$ is 1 if true and 0 if false. Softmax is not needed to predict, because it never changes which score is largest. The confusion matrix counts $C_{ij} = \#\{\text{pictures with true digit } i \text{ predicted as } j\}$; the diagonal holds the correct answers, and accuracy is $\operatorname{tr}(C)/\sum_{ij} C_{ij}$. `train.py & report.py · evaluate, confusion_matrix`

10.4 How precise is a test accuracy?

Treat each test picture as a coin flip that comes up "correct" with probability a. The standard error of the measured accuracy over N pictures is

$$\text{SE} = \sqrt{\frac{a(1-a)}{N}} = \sqrt{\frac{0.985 \times 0.015}{271}} \approx 0.0074,$$

about ± 0.7 percentage points, and one picture is worth $1/271 \approx 0.37$ points. Run-to-run variation from the random seed adds to this: seeds 1 to 5 ranged from 97.4% to 99.6%. A change only counts as an improvement if it beats that spread over several seeds.

11 One-page summary

Concept	Formula	Code
Scaling	$x \leftarrow x/16$	`data.py`
One-hot	$t_k = [\,k = y\,]$	`one_hot`
Dense forward	$O = XW + \mathbf{b}$	`Dense.forward`
He init	$W_{ij} \sim \mathcal{N}(0,\ 2/n_{\text{in}})$, i.e. std. dev. $\sqrt{2/n_{\text{in}}}$	`Dense.__init__`
Dense backward	$\frac{\partial L}{\partial W} = X^{\mathsf{T}}G, \quad \frac{\partial L}{\partial \mathbf{b}} = \sum_{\text{rows}} G, \quad \frac{\partial L}{\partial X} = GW^{\mathsf{T}}$	`Dense.backward`
ReLU	$\max(0, u)$; backward $G \odot [\,U > 0\,]$	`ReLU`
Dropout	$\frac{m}{q}x,\ m \sim \text{Bernoulli}(q)$; identity at test time	`Dropout`
Softmax	$p_k = e^{z_k - \max z} \big/ \sum_j e^{z_j - \max z}$	`softmax`
Cross-entropy	$L = -\frac{1}{n}\sum_i \log p^{(i)}_{y^{(i)}}$ (blind guess: $\ln 10$)	`cross_entropy`
Loss gradient	$\frac{\partial L}{\partial Z} = \frac{1}{n}(P - T)$	`cross_entropy`
Chain rule	$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial O}\,\frac{\partial O}{\partial U}$, applied last layer to first	`Network.backward`
Gradient check	$\frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}$, error $O(\varepsilon^2)$	`gradcheck.py`
SGD	$w \leftarrow w - \eta g$; stable only if $\eta < 2/a$	`SGD.step`
Momentum	$v \leftarrow \mu v - \eta g,\ w \leftarrow w + v$; speed-up $\frac{1}{1-\mu}$	`SGD.step`
Adam	$w \leftarrow w - \eta\,\hat{m}/(\sqrt{\hat{v}} + \epsilon),\ \hat{m} = \frac{m}{1-\beta_1^t},\ \hat{v} = \frac{v}{1-\beta_2^t}$	`Adam.step`
Prediction	$\hat{y} = \arg\max_k z_k$	`Network.predict`
Accuracy	$\frac{1}{N}\sum_i [\,\hat{y}^{(i)} = y^{(i)}\,] = \text{tr}(C)/N$	`evaluate`
Early stopping	keep $w^{(e^\star)}$, $e^\star = \arg\min_e L^{(e)}_{\text{val}}$	`fit`
Uncertainty	$\text{SE} = \sqrt{a(1-a)/N} \approx 0.7$ points here	--